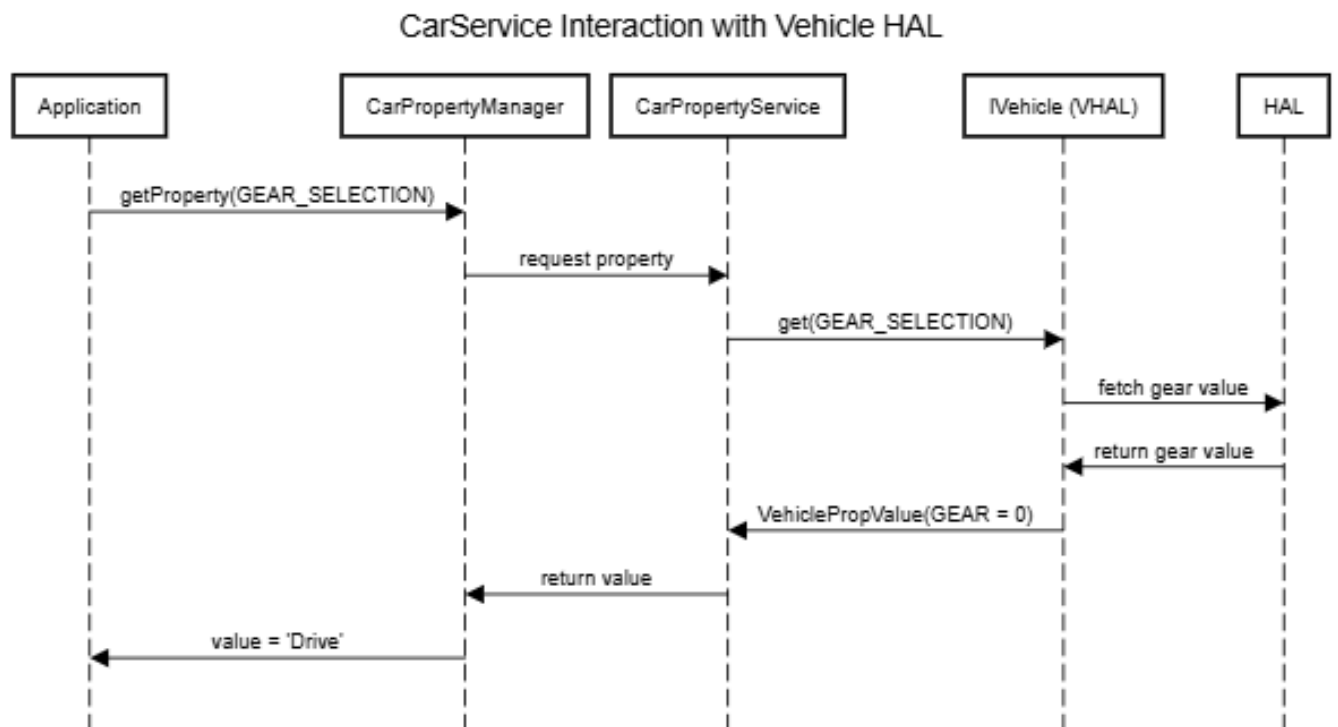


CarService Interaction with Vehicle HAL

CarService acts as the central system service in Android Automotive, bridging applications and the Vehicle HAL (VHAL). The Vehicle HAL exposes vehicle-specific properties such as gear, ignition state, HVAC, and speed through the IVehicle AIDL/HIDL interface. The VHAL interface is defined in IVehicle.aidl ([hardware/interfaces/automotive/vehicle/aidl/](#)) or HIDL in older versions. CarService and its sub-services (CarPropertyService, CarPowerService, etc.) call into IVehicle to read, write, or subscribe to vehicle properties.

1. Sequence diagram from application to VHAL



Sequence Explanation: Reading Gear from VHAL

1. App calls → `CarPropertyManager.getProperty(GEAR_SELECTION)`.
2. CarPropertyManager calls into → ICarImpl binder.
3. ICarImpl routes to → CarPropertyService.
4. CarPropertyService calls → `mVehicle.get(VehiclePropValue)` on VHAL.
5. VHAL returns the current gear → response flows back to the app.

Methods Called on Vehicle HAL (IVehicle)

1. getAllPropConfigs()

- Used by CarPropertyService during initialization.
- Fetches all supported vehicle property configurations from VHAL.
- Example properties: gear, ignition state, HVAC, speed.

Inside CarPropertyService:

```
List<VehiclePropConfig> configs = mVehicle.getAllPropConfigs();
```

Inside IVehicle:

```
/**
 * Returns a list of all property configurations supported by this vehicle
 * HAL.
 */
getAllPropConfigs() generates (vec<VehiclePropConfig> propConfigs);
```

2. getPropConfig(int propertyId)

- When an app (via CarPropertyManager) requests information about a specific vehicle property (not all configs, just one).
- CarPropertyService checks if the property is already cached. If not cached, it calls into IVehicle → getPropConfig(propertyId) to fetch metadata such as:
Access type (read/write), Change mode (static, on-change, continuous), Min/max values (for ranges like speed, HVAC temperature).

Inside CarPropertyService:

```
VehiclePropConfig config = mVehicle.getPropConfig(propertyId);
```

```
mPropConfigs.put(propertyId, config);
```

3. get(VehiclePropValue request)

- Reads the current value of a property from VHAL.
- Example: get the current gear, ignition state, or speed.
- Called like: *VehiclePropValue value = mVehicle.get(request);*

Inside IVehicle:

```
/**
 * Get a vehicle property value.
 *
 * For VehiclePropertyChangeMode::STATIC properties, this method must always
 * return the same value always.
 * For VehiclePropertyChangeMode::ON_CHANGE properties, it must return the
 * latest available value.
 *
 * Some properties like RADIO_PRESET requires to pass additional data in
 * GET request in VehiclePropValue object.
 *
 * If there is no data available yet, which can happen during initial stage,
 * this call must return immediately with an error code of
 * StatusCode::TRY_AGAIN.
 */
get(VehiclePropValue requestedPropValue)
generates (StatusCode status, VehiclePropValue propValue);
```

4. set(VehiclePropValue value)

- Writes a value into VHAL.
- Example: turn headlights ON, set HVAC temperature, control windows.
- Typically invoked by CarPropertyService when an app changes a property.

```
/**
 * Set a vehicle property value.
 *
 * Timestamp of data must be ignored for set operation.
 *
 * Setting some properties require having initial state available. If initial
 * data is not available yet this call must return StatusCode::TRY_AGAIN.
 * For a property with separate power control this call must return
 * StatusCode::NOT_AVAILABLE error if property is not powered on.
 */
set(VehiclePropValue propValue) generates (StatusCode status);
```

5. subscribe(IVehicleCallback callback, int[] propIds, int sampleRate)

- Subscribes to updates for one or more vehicle properties.
- Example: speed updates every 100 ms.
- Apps indirectly use this via CarPropertyManager → CarPropertyService → VHAL.
- VHAL pushes updates back via the callback.

```
/**
 * Subscribes to property events.
 *
 * Clients must be able to subscribe to multiple properties at a time
 * depending on data provided in options argument.
 *
 * @param listener This client must be called on appropriate event.
 * @param options List of options to subscribe. SubscribeOption contains
 *               information such as property Id, area Id, sample rate, etc.
 */
subscribe(IVehicleCallback callback, vec<SubscribeOptions> options)
    generates (StatusCode status);
```

6. unsubscribe(IVehicleCallback callback, int[] propIds)

- Unsubscribes from previously subscribed vehicle property updates.

```
/**
 * Unsubscribes from property events.
 *
 * If this client wasn't subscribed to the given property, this method
 * must return StatusCode::INVALID_ARG.
 */
unsubscribe(IVehicleCallback callback, int32_t propId)
    generates (StatusCode status);
```

7. dump() / debug()

- Used internally for debugging/logging state of the VHAL.
- Called via adb shell dumpsys car_service.

```
/**
 * Print out debugging state for the vehicle hal.
 *
 * The text must be in ASCII encoding only.
 *
 * Performance requirements:
 *
 * The HAL must return from this call in less than 10ms. This call must avoid
 * deadlocks, as it may be called at any point of operation. Any synchronization
 * primitives used (such as mutex locks or semaphores) must be acquired
 * with a timeout.
 */
debugDump() generates (string s);
```

Conclusion:

CarService relies on the Vehicle HAL (IVehicle) to interact with vehicle hardware. The key methods are getAllPropConfigs(), getPropConfig(), get(), set(), subscribe(), unsubscribe(), and debug(). Different CarService sub-services call these methods depending on their domain (property, power, input, audio, etc.). This abstraction ensures that Android Automotive applications can access vehicle data through a stable API without dealing directly with hardware.

